

Сравнительный тест производительности JavaScript, Python и C++ на стандартных платформенных типах данных

Методика замеров, окружение теста

Тестовое окружение: создано на основе тестового проекта с набором сервисных модулей, которые позволяют исполнять JS, Python и C++ код.

Тестовый сценарий: посредством Invoke вызывается метод БЛ, который внутри облака проводит замеры выполнения сценария, вызывает выполнение теста на JS, Python или C++, а также фиксирует результаты теста в файл.

Тестовые данные: «RecordSet» (далее по тексту «рекордсет»), который состоит из 620 элементов с разными типами данных:

- логические значения;
- целочисленные значения размером 1, 2 и 4 байт;
- числа с двойной и одинарной точностью;
- строки длиной около 45 символов (исключают влияние Small String Optimizations);
- массивы дат размером в 50 элементов.

Полученную выборку требовалось обработать любым образом в целевом языке и выдать усредненное значение на десять тысяч вызовов тестового кода.

Настройка оптимизации: библиотека с кодом для тестового облака была скомпилирована со всеми оптимизациями (флаг -O3 в GCC), а на тестовом облаке было выключено логирование. Других мер для уменьшения влияния побочных процессов на тестирование не предпринималось, т.к. был важен порядок выполнения кода — микросекунды, миллисекунды, и прочее.

C++

Для теста рекордсет передавался как ссылка.

Тип замера	Длительность, мс
Полный перебор рекордсета внутри C++	1.798
Без ответа	1.801
С ответом	1.800

Внутри C++ замеров не проводилось.

Python

Для теста рекордсет передавался как ссылка.

Тип замера	Длительность, мс
Полный перебор рекордсета внутри Python	22.3
Без ответа	22.34
С ответом	22.2

Внутри Python замеров не проводилось.

JavaScript

На момент проведения теста отсутствовала часть кода, специфичного для рекордсетов внутри серверного JS Wasaby Framework, в связи с чем пришлось подключать через библиотеку RequireJS дополнительные зависимости от интерфейсной платформы WS.Data.

Соответственно, результирующий JS-код выполнялся асинхронно. В связи с тем, что отсутствует «мост» между JS и C++, тесты JS так же включали в себя замеры сериализации-десериализации.

Для теста рекордсет передавался как значение: сначала его сериализовали в JSON, затем передавали в серверный JS, потом десериализовали в рекордсет и выполняли тест.

Тип замера	Длительность, мс
Полный перебор рекордсета внутри JS	39
Вместе с вызовом обёрток над V8 из C++	83
Всё время внутри C++, включая сериализацию и десериализацию рекордсета	97

Результаты теста с рекордсетом, в который включен массив дат.

Тип замера	Длительность, с
Полный перебор рекордсета внутри JS	1.48
Вместе с вызовом обёрток над V8 из C++	1.58
Всё время внутри C++, включая сериализацию и десериализацию рекордсета,	1.616

Ниже представлен соответствующий фрагмент логов:

```
1 Large recordset operations: Large Rset From JS time spent inside JS
  avg: 39.821000 m sec, time inside C++ avg: 83.140000 msec, time outside
  a vg:
2 97.718000 msec, over 1000 iterations. Large Rset From JS with
  ArrayTimeime spent inside JS avg: 14 81.460000 msec, time inside C++
  avg:
3 1580.400000 msec, time outside avg: 1616.970000 msec, over 100
  iterations.
```

Причину сильного замедления сказать сложно ввиду общей сложности конструкции, применявшейся для тестирования.